

Phase 3

RAG /ask Endpoint

Ollama Cloud · gpt-oss:20b-cloud · Anti-hallucination prompt · Confidence guard · Two-layer refusal · Cited sources

Today's win in one line: The ingestion engine now answers questions. A single `POST /ask` call retrieves the top matching chunks by cosine similarity, refuses to invoke the LLM if none clear a confidence threshold, hands the survivors to `gpt-oss:20b-cloud` under a strict "answer only from these passages" system prompt, and returns a grounded JSON answer with page-level citations. Hallucinations blocked at two independent layers, verified with a live "capital of France" test that correctly returned zero sources over HTTP.

What you built / verified	How
Ollama Cloud account + API key	Signed up, ran <code>ollama signin</code> , model <code>gpt-oss:20b-cloud</code> reachable from the local Ollama daemon on free tier.
LLM client (<code>app/llm/llm_client.py</code>)	Thin wrapper over Ollama's <code>/api/chat</code> with a 120s timeout and <code>think: false</code> for the reasoning model.
Answer service (<code>app/services/answer_service.py</code>)	Glue: search Qdrant → confidence guard @ 0.45 → build prompt → call LLM → return <code>{question, answer, sources[]}</code> .
Anti-hallucination system prompt	Strict rules: only use the provided passages, cite by page, refuse if no answer is present.
Confidence threshold pattern (0.45)	If the top retrieved chunk scores below 0.45, the LLM is never called — a fixed refusal string is returned instead.
Two-layer refusal (belt + suspenders)	Layer 1: code guard skips the LLM. Layer 2: system prompt tells the model to refuse when passages don't contain the answer.
Ask route (<code>app/api/routes_ask.py</code>)	FastAPI POST endpoint with Pydantic request/response models; wired into <code>main.py</code> , app version bumped to 0.3.0.
Verified end-to-end (3 tests)	"Why volumes in Docker" and "how do containers keep data across restarts" — grounded answers with cited pages. "Capital of France" — refusal with empty sources over HTTP.

1 Ollama Cloud & gpt-oss:20b-cloud

1.1 What Ollama Cloud is, in plain words

Ollama Cloud is Anthropic-style cloud hosting *for Ollama models*. Same Ollama CLI, same `/api/chat` and `/api/generate` endpoints, same JSON shape — but the model runs on their GPUs instead of your CPU. Your local Ollama daemon acts as a proxy: you send requests to `localhost:11434` as usual, and if the model name ends in `-cloud`, the daemon forwards it to Ollama's servers using the API key you signed in with.

DEFINITION	A managed inference service for Ollama models, accessed transparently through the local Ollama daemon.
WHERE USED	Any time your machine can't realistically run a 20B-parameter model. A 20B model needs ~13 GB just to load — your i7 with 16 GB RAM would swap to disk and die.
EXAMPLE	Running <code>ollama signin</code> once, then <code>ollama run gpt-oss:20b-cloud</code> — the prompt goes out over HTTPS to Ollama's GPU, response comes back like it ran locally.
KEY TERMS	<code>ollama signin</code> (auth), <code>-cloud</code> suffix (routes to cloud), API key (stored by daemon), free tier (limited req/min).
WHY FOR INTERNSHIP	Lets you demo a serious LLM inside a laptop project without paying OpenAI/Anthropic API bills and without shipping the model. Same code path works when you swap to a bigger cloud model later.

1.2 Why gpt-oss:20b-cloud specifically

- **Open-weight** — no vendor lock. If Ollama Cloud disappears, you can host the same weights yourself on a rented GPU.
- **Big enough to follow instructions** — 20B params is the sweet spot for "actually respects a system prompt saying refuse when the context doesn't answer." Smaller models (3B/7B) will happily invent answers even when told not to.
- **Reasoning-tuned** — it's a reasoning model, which means it thinks before speaking. That's a superpower *and* a trap; see §3.
- **Free tier is enough for a demo** — you don't need billing set up to prove the pipeline works.

YOU PROVED THIS TODAY

Same `ChatClient` code that could talk to a local llama3 can now talk to a 20B cloud model just by changing `LLM_MODEL` in `.env`. That's the payoff of using Ollama as a uniform interface.

2 Chat vs Generate endpoints

2.1 The two Ollama endpoints and when to use each

Ollama exposes two very similar endpoints. Picking the wrong one is a small mistake with big consequences.

Endpoint	Input shape	When to use
<code>/api/generate</code>	A single string <code>prompt</code> .	One-shot text completion. No concept of roles. Useful for "continue this text" tasks.
<code>/api/chat</code>	A list of messages, each with a <code>role</code> (<code>system</code> , <code>user</code> , <code>assistant</code>) and <code>content</code> .	Anything that needs a system prompt or multi-turn context. This is what real apps use.

DEFINITION	<code>/api/generate</code> is "raw completion," <code>/api/chat</code> is "role-structured conversation."
WHERE USED	You want <code>/api/chat</code> anywhere you have a system prompt — because that's the only way to <i>send</i> a system prompt cleanly. RAG apps almost always use <code>/api/chat</code> .
EXAMPLE	Your <code>llm_client.py</code> POSTs a body like <code>{"model": "gpt-oss:20b-cloud", "messages": [{"role": "system", "content": "..."}, {"role": "user", "content": "..."}], "stream": false}</code> .
KEY TERMS	Roles (<code>system</code> / <code>user</code> / <code>assistant</code>), <code>messages</code> array, <code>stream</code> , response field <code>message.content</code> .
WHY FOR INTERNSHIP	The interviewer will absolutely ask "what's a system prompt and how do you send one." <code>/api/chat</code> is the answer.

TRAP

Beginners reach for `/api/generate` because it feels simpler. Then they concatenate the system prompt into the user prompt as a string. The model treats that as user text and follows it much less strictly than a real `system` role. Don't do that.

3 The think:false flag on reasoning models

3.1 What "reasoning models" actually do

A reasoning model is trained to produce a hidden internal monologue before the final answer. In the response payload that shows up as a separate `thinking` field (or gets streamed as reasoning tokens) — you don't see it in the UI, but it exists and it costs tokens and latency.

DEFINITION	A model tuned to plan-then-answer instead of answer-directly. The plan is called "thinking" and is usually hidden from the end user.
WHERE USED	Math, code, and multi-step reasoning tasks where thinking-out-loud measurably improves accuracy.
EXAMPLE	<code>gpt-oss:20b-cloud</code> , OpenAI's o-series, DeepSeek-R1. Contrast with plain llama3, which just answers.
KEY TERMS	Thinking tokens, reasoning tokens, chain of thought, <code>think</code> flag.
WHY FOR INTERNSHIP	Reasoning is why 20B feels smarter than 20B ought to feel. But you have to know when to turn it off.

3.2 Why you set `think:false`

In a RAG system, the model isn't solving a math problem — it's *copying from a passage*. The retrieved chunks already contain the answer; the model just needs to phrase it and cite it. Reasoning tokens on top of that add:

- **Latency** — every thinking token is a forward pass. For a copy-from-context task, this is pure wait time.
- **Cost** — even on free tier, thinking counts toward rate limits.
- **Answer contamination** — sometimes the thinking leaks into the visible answer. Users get "Let me think... okay, based on Passage 1... hmm actually let me reconsider..." instead of the clean sentence they wanted.

Passing `think: false` in the chat options tells the model to skip the reasoning phase and answer directly. It's the right default for RAG.

YOU PROVED THIS TODAY

The volumes-in-Docker answer came back as a clean, cited sentence — no visible planning prose. That's `think:false` doing its job.

TRAP

Don't turn thinking off for every use case. If you later build a code-fixing agent or a multi-hop question decomposer, you want thinking *on*. The rule is: turn thinking off when the answer is already in the context; keep it on when the model has to figure something out.

4 The anti-hallucination system prompt

4.1 What a system prompt is

The system prompt is the first message in the `messages` array with role `system`. It's how you set the model's rules before the user says anything. Models are trained to weight it heavier than user messages — it's the closest thing to giving the LLM a job description.

DEFINITION	A pre-conversation instruction that sets the model's persona, constraints, and refusal policy.
WHERE USED	Any production LLM app. Bare chat with no system prompt gets you a very different model than you think you have.
EXAMPLE	Yours says (paraphrased): "You are a document assistant. Answer using ONLY the passages below. If the answer isn't there, reply exactly 'I could not find the answer in the provided documents.' Cite passages by their page label."
KEY TERMS	Role <code>system</code> , prompt discipline, refusal string, citation format.
WHY FOR INTERNSHIP	System prompt discipline is the difference between a demo that hallucinates and one that doesn't.

4.2 The three rules your prompt enforces

1. **Only use the provided passages.** No world knowledge, no guessing. If a passage says the sky is green, the answer is "the sky is green." Grounding beats correctness — a system that echoes the source is auditable; a system that quietly overrides it is not.
2. **Refuse with a fixed string when the answer isn't present.** A fixed refusal string is a machine-readable signal — your caller can check `sources == []` or the exact refusal text and branch. A free-form "I don't know" isn't.
3. **Cite passages by page.** Every claim should be traceable back to a chunk. That's the whole point of RAG.

PROMPT DISCIPLINE IN ONE LINE

Every rule in the system prompt exists because you don't trust the model. Assume it will violate whatever you don't explicitly forbid.

5 The confidence threshold pattern (0.45)

5.1 What "confidence" means here

Qdrant returns a cosine similarity score for every retrieved chunk — a number between roughly 0 and 1 measuring how close the question's embedding is to the chunk's embedding. It is *not* a probability that the answer is correct; it's a semantic-similarity score.

DEFINITION	The top retrieved chunk's cosine similarity to the query embedding. Higher = closer in meaning.
WHERE USED	Every RAG system needs some notion of "did retrieval actually find anything?" — cosine score is the cheapest, most direct signal.
EXAMPLE	Your Day 2 test: "revenue Q2" vs "income last quarter" = 0.72 (strong). Same query vs an unrelated weather sentence = 0.45. That gap is why 0.45 works as a floor.
KEY TERMS	Cosine similarity, top-k, threshold, retrieval floor, semantic gap.
WHY FOR INTERNSHIP	"How do you know retrieval worked?" is one of the top three RAG interview questions.

5.2 How you picked 0.45

It's calibrated, not magic. During Day 2 you measured:

- **Semantically related** queries against your corpus: ~0.55 – 0.75.
- **Unrelated** queries ("capital of France," "weather today"): ~0.20 – 0.40.

0.45 sits in the valley between those distributions — high enough that unrelated queries fall below it, low enough that a rephrased-but-valid query still passes. It's a project-specific number: change corpus or embedding model and you re-calibrate.

TRAP

Don't hardcode the threshold in the service. Put it in `.env` or config so you can tune it without redeploying. A future Day 4 improvement.

6 Two-layer refusal (belt + suspenders)

6.1 Why one layer isn't enough

Either layer alone has a failure mode. Together they cover each other.

Layer	How it fires	What it catches	What it misses
1. Code guard (in <code>answer_service.py</code>)	Top score < 0.45 → skip LLM, return fixed refusal.	Obvious misses. Cheap and fast — saves tokens, saves latency.	Cases where retrieval brings back <i>almost</i> - relevant chunks (score 0.50) that don't actually contain the answer.
2. Prompt guard (in the system prompt)	Model reads passages, sees no answer, returns the fixed refusal string.	The 0.50-but- irrelevant case above. The model can tell the passages don't actually answer the question.	Models occasionally ignore instructions and hallucinate anyway — rare but real.

YOU PROVED THIS TODAY

The "capital of France" test hit layer 1 — the top chunk was so far off that the LLM was never called. Sources came back empty, refusal string came back verbatim. If you'd asked something adjacent (like "What is Docker Swarm?" against a Docker docs corpus that only covers Compose), you'd probably have seen layer 2 fire instead.

6.2 Why "belt and suspenders" is the right mental model

Belt alone fails when it stretches. Suspenders alone fail when they snap. Both together — you'd have to have two independent failures on the same day. That's the whole engineering pattern: **defense in depth**. It's the same reason you check `EMBEDDING_DIM = 768` in three places (config, embedding client, Qdrant store) — layered checks turn silent bugs into loud failures.

7 The /ask endpoint end-to-end

7.1 The full request lifecycle

Walk through it once so you can narrate it in the interview without hesitating.

1. **HTTP request lands.** `POST /ask` with JSON `{"question": "...", "top_k": 3}`. FastAPI validates it against the Pydantic `AskRequest` model — bad input returns 422 before your code runs.
2. **Route delegates to service.** `routes_ask.py` is thin — it calls `answer_service.answer(question, top_k)`. Business logic lives in the service, not the route. Same pattern as Day 2's ingestion service.
3. **Retrieval.** Service embeds the question with the same `nomic-embed-text` model used at ingest time (same model → same vector space → similarity means something). Qdrant returns the top-k chunks with scores and payloads.
4. **Confidence guard.** If the top score < 0.45 , return `{question, "I could not find the answer in the provided documents.", []}` immediately. LLM never called.
5. **Prompt build.** Format the surviving chunks into "Passage 1 — file.pdf p.3: [text]" blocks. Prepend the system prompt.
6. **LLM call.** `llm_client.chat(messages, think=False, timeout=120)`. 120s is generous — cloud reasoning models can be slow on the first call.
7. **Response assembly.** Return `{question, answer, sources[]}` where each source has `file_id`, `file_name`, `page_number`, `chunk_index`, `score`, `label`. The label ("Day1_Study_Guide.pdf — p.3") is what a UI would show as a clickable citation.

WHY THE RESPONSE SHAPE MATTERS

Returning `sources[]` as structured data (not just as text inside the answer) is what makes this callable from another program. n8n could take the answer and post it to Slack, and separately post the source list as buttons. That's Phase 6.

Q&A Likely interview questions on today's work

Answers in your voice — the way you'd actually explain it, not a textbook.

Question

Your answer

What is RAG in one sentence?

It's an LLM that answers using text I fetched for it from a database instead of using only what it memorized during training. My app retrieves relevant chunks from Qdrant first, then hands them to the model to phrase the answer.

Question	Your answer
Why RAG instead of just fine-tuning?	Because my documents change. Fine-tuning bakes knowledge into the weights, so every new PDF means a new training run. RAG keeps the model fixed and just updates the vector store — I can add a document and it's immediately queryable, no retraining.
What is hallucination and why do LLMs do it?	Hallucination is when the model produces something that sounds right but isn't in the source. It happens because LLMs are trained to produce plausible-sounding text, not truthful text — if it doesn't know the answer, it will still generate a fluent guess unless something stops it.
How do you prevent hallucination in your system?	Two layers. First, a confidence threshold — if the best retrieved chunk isn't similar enough to the question (score under 0.45), I don't call the LLM at all, I return a fixed refusal. Second, the system prompt tells the model to only answer using the passages I gave it, and to return that same refusal string if the passages don't contain the answer.
Why the number 0.45 for your threshold?	I measured it. In Day 2 I ran related queries against my corpus and got scores around 0.55 to 0.75, and unrelated queries scored around 0.20 to 0.40. 0.45 sits in the gap — high enough that unrelated stuff fails, low enough that a rephrased valid question still passes. It's calibrated to my embedding model and my corpus, not universal.
Why two layers of refusal instead of one?	Defense in depth. The code guard catches the obvious cases cheaply — no tokens spent, no latency. But if retrieval brings back a chunk that's <i>almost</i> relevant (say 0.50) that doesn't actually contain the answer, the code guard passes and the prompt guard catches it. Either alone has a failure mode; together they cover each other.
Walk me through what happens when someone asks a question your docs can't answer.	The question gets embedded and searched against Qdrant. Qdrant returns the top chunk anyway — vector search always returns something. But the top score comes back low, like 0.28 for "capital of France" against a Docker corpus. My service sees that's under 0.45, skips the LLM entirely, and returns the refusal string with an empty sources array. I tested this end-to-end today.
Why did you pick gpt-oss: 20b-cloud?	Three reasons. It's open-weight so I'm not locked to a vendor. It's big enough (20 billion params) to actually respect the system prompt — smaller models will happily hallucinate even when told not to. And Ollama's free tier covers demo use, so I didn't need to set up billing to prove the pipeline.
Why run it in the cloud instead of locally?	My laptop is 16 GB RAM, i7 CPU-only. A 20B model needs about 13 GB just to load and would swap to disk during inference. Cloud runs it on their GPU and my local Ollama daemon just proxies the request. Same API, same code, different execution location.

Question	Your answer
What's the difference between <code>/api/chat</code> and <code>/api/generate</code> in Ollama?	Generate takes one raw prompt string. Chat takes a list of messages, each with a role — system, user, or assistant. I use chat because it's the only clean way to send a system prompt with a real <code>system</code> role. Concatenating the system prompt into the user string works but the model respects it less.
What does <code>think:false</code> do?	<code>gpt-oss:20b-cloud</code> is a reasoning model — it produces hidden "thinking" tokens before the final answer. That's great for math or coding, but for RAG the answer is already in the passages, the model just has to phrase it. Reasoning adds latency and cost and sometimes leaks planning prose into the visible answer. So I turn it off for this use case.
What's a system prompt and why does it matter here?	It's the first message with role "system" — the model reads it before it sees any user input, and it's trained to weight it heavier than user messages. It's how I tell the model "you're a document assistant, only use the passages I give you, refuse with this exact phrase if you can't answer." Without it, the model uses its full world knowledge and my confidence guard is the only defense.
How does your app cite sources?	The response includes a <code>sources</code> array. Each entry has the file name, page number, chunk index, similarity score, and a human-readable label like "Day1_Study_Guide.pdf — p.3". A UI can render those as clickable citations that jump to the source PDF page.
What are the risks of your current approach?	Three I'd flag. The threshold is hardcoded — if I swap the embedding model or corpus, I have to re-tune it. Retrieval quality caps answer quality; if chunking is bad, the model gets bad context and answers badly. And the system prompt can still be ignored occasionally — the model isn't guaranteed to refuse, it's just heavily nudged.
Walk me through the <code>/ask</code> endpoint end-to-end.	FastAPI receives the POST, validates the JSON against my Pydantic model. The route calls <code>answer_service.answer</code> . The service embeds the question with the same model I used at ingest time, searches Qdrant for the top 3 chunks, and checks if the top score is above 0.45. If not, it returns the refusal immediately. If yes, it builds a prompt that lists the passages with their page labels, calls Ollama with a strict system prompt and <code>think:false</code> , gets a grounded answer back, and returns a JSON with the question, the answer, and the source list.

Day 3 · Phase 3 complete. RAG `/ask` endpoint working end-to-end with confidence guard and cited sources. Next: Phase 4 — OCR path in the PDF parser for scanned and image-based pages using Tesseract.